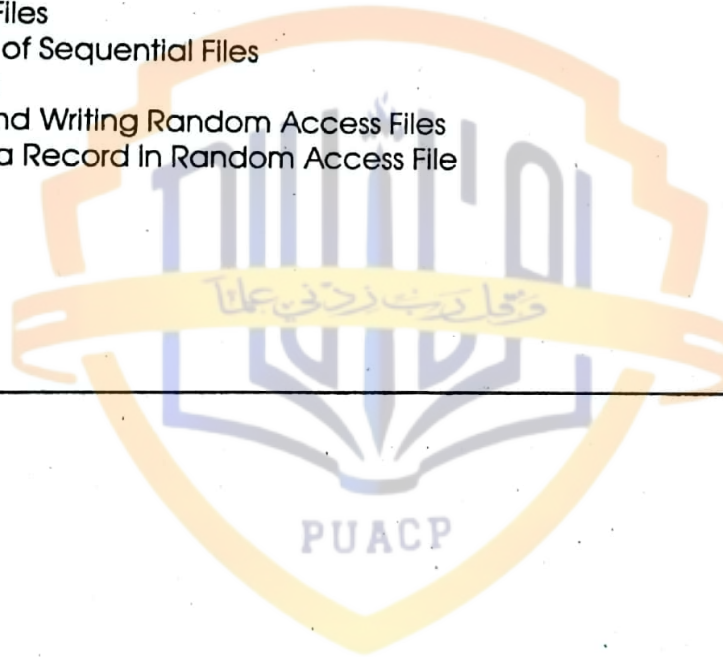


Chapter Overview

- 13.1 File Handling
- 13.2 File System Object
- 13.3 File Object
- 13.4 Folder Object
- 13.5 Drive Object
- 13.6 TextStream Object
- 13.7 Creating Sequential Files
- 13.8 Reading Sequential Files
 - 13.8.1 Drawback of Sequential Files
- 13.9 Random Access Files
 - 13.9.1 Reading and Writing Random Access Files
 - 13.9.2 Searching a Record In Random Access File

Short Questions
Multiple Choices
Fill in the Blanks
True/False



13.1 File Handling

A file is a group of related records. It is used to store data and information. You can save any type of data in files. There are different ways of dealing with files in Visual Basic. Two important types of files are as follows:

- Sequential Files
- Random Access Files

Visual Basic provides **File System Object (FSO)** that is used to manipulate files, folders and drives of computer system. The user can read and write different sequential files. The following steps are required before using **File System Object**:

- Include **Microsoft Scripting Runtime Library** in project by selecting **References** from **Project** menu.
- Create the reference at runtime by using the following method:
`CreateObject("Scripting.FileSystemObject")`

Different types of object in file system are as follows:

- File System Object (FSO)
- File Object
- Folder Object
- Drive Object
- TextStream Object

13.2 File System Object

File system object is used to manipulate files, folder and drives by using its different built-in methods. Some important methods of this object are as follows:

Method	Description
CopyFile	Used to copy an existing file
CopyFolder	Used to copy an existing folder
CreateFolder	Used to create a folder
CreateTextFile	Used to create a text file
DeleteFile	Used to delete a file
DeleteFolder	Used to delete a folder
DriveExists	Used to determine whether a drive exists or not
FileExists	Used to determine whether a file exists or not
FolderExists	Used to determine whether a folder exists or not
GetAbsolutePathName	Used to return full path
GetDrive	Used to return a specified drive
GetDriveName	Used to return drive name
GetFile	Used to return a specified file
GetFileName	Used to return the file name
GetFolder	Used to return a specified folder
GetParentFolderName	Used to return the name of parent folder
GetTempName	Used to create and return a string representing file name
MoveFile	Used to move a file
MoveFolder	Used to move a folder
OpenTextFile	Used to open a text file

13.3 File Object

File object is used to access any file on computer. Different properties of File object are:

Property	Description
Attributes	Used to read or set file attributes
DateCreated	Used to determine the date on which the file was created
DateLastAccessed	Used to determine the date on which the file was last accessed
DateLastModified	Used to determine the date on which the file was last modified
Drive	Used to determine the drive letter on which the file exists
Name	Used to set or get the name of the file
ParentFolder	Used to return the parent folder of the file
Path	Used to return the path of a specified file
Size	Used to determine the size of file
Type	Used to determine the type of file

Some important methods of File object are as follows:

Method	Description
Copy	Used to copy a file
Delete	Used to delete a file
Move	Used to move a file
OpenAsTextStream	Used to open a file. A file can be opened for reading, writing or appending

Program 13.1

Write a program that gets a file name from user and displays its attributes on form.

1. Start a new Standard EXE project and design the form as follows:



2. Select **References...** from **Project** menu. A dialog box will appear.
3. Select **Microsoft Scripting Runtime** and click **OK**.
4. Add the following code in **Click** event of the button.

```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1_Click()
    On Error GoTo FError
    Dim FSO As New FileSystemObject
    Dim f As File
    Dim Info As String
    Set f = FSO.GetFile(Text1.Text)
    Print "File Name: " & f.Name
    Print "Created: " & f.DateCreated
    Print "Modified: " & f.DateLastModified
    Print "Accessed: " & f.DateLastAccessed
    Exit Sub

FError:
    MsgBox Err.Description
End Sub

```

5. Run the project.
6. Enter any file name with path and click the button. If the given file exists, its attributes will be displayed on the form. Otherwise an error message will appear.

13.4 Folder Object

Folder object is used to represent a folder on a disk. You can manipulate folders by using different properties and methods of Folder object. Following are some important properties of Folder object.

Property	Description
Attributes	Used to return or set the attributes of folder
DateCreated	Used to determine the date on which folder was created
DateLastAccessed	Used to determine the date on which folder was last accessed
DateLastModified	Used to determine the date on which folder was last modified
Drive	Used to determine the drive on which the folder exists
ParentFolder	Used to determine the parent folder of the folder
IsRootFolder	Used to determine if the folder is a root folder or not
Name	Used to return the name of the folder
Path	Used to determine the path of the folder
Size	Used to determine the size of the folder

13.5 Drive Object

Drive object is used to manipulate the drives of computer system. Some important properties of Drive object are as follows:

Property	Description
AvailableSpace	Used to determine the available space on the drive
DriveLetter	Used to determine the drive letter
DriveType	Used to determine the type of drive
FileSystem	Used to determine the file system used on the drive
FreeSpace	Used to determine the free space on the drive
RootFolder	Used to determine the root folder of the drive
TotalSize	Used to determine the total size of the drive

13.6 TextStream Object

TextStream object is used to read and write sequential files. Some important properties of this object are as follows:

Property	Description
AtEndOfLine	Used to determine whether the position is at the end of line or not
AtEndOfStream	Used to determine whether the position is at the end of stream or not
Column	Used to determine the position of current character in the line
Line	Used to determine the current line number

Some important methods of TextStream object are as follows:

Method	Description
Close	Used to close the TextStream object
Read	Used to read the specified number of characters in the TextStream
ReadAll	Used to read all contents of TextStream
ReadLine	Used to read a line of the TextStream
Skip	Used to skip specified number of character in the TextStream
SkipLine	Used to skip a line in the TextStream
Write	Used to write to the TextStream
WriteBlankLines	Used to write given number of blank lines in the TextStream
WriteLine	Used to write a line and then end-of-line character

13.7 Creating Sequential Files

You can create a sequential file to store your data and information. In order to create and manipulate text files, you need to take the following steps:

- Create a FileSystemObject
- Create a File Object
- Create a TextStream Object

First of all, FileSystemObject is created to create new files on the disk. File object is created to link this object with the physical file on the disk. TextStreamObject is then used to read or write the contents of the file.

Program 13.2

Write a program that inputs first name, second name and email in text boxes and stores this data in a sequential file.

1. Start a new Standard EXE project and design the form as follows:

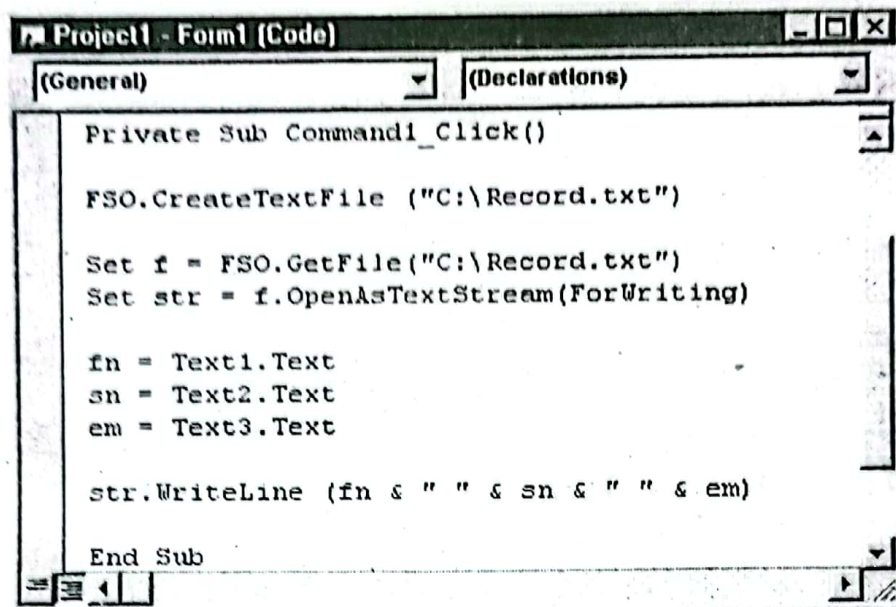
2. Add the following code in General declaration section.

```

Project1 - Form1 (Code)
(General) (Declarations)
Dim FSO As New FileSystemObject
Dim f As file
Dim str As TextStream
Dim fn As String, sn As String, em As String

```

3. Add the following code in Click event of the button.



```

Project1 - Form1 (Code)
(General) (Declarations)

Private Sub Command1_Click()

    FSO.CreateTextFile ("C:\Record.txt")

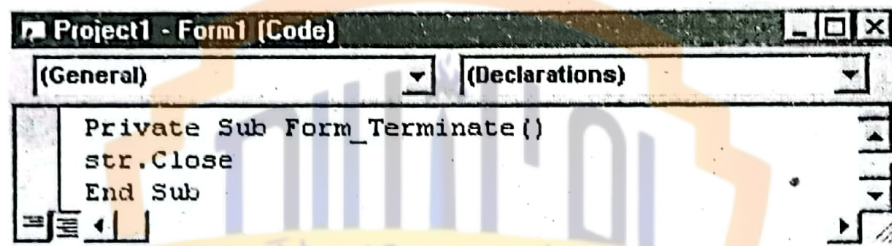
    Set f = FSO.GetFile("C:\Record.txt")
    Set str = f.OpenAsTextStream(ForWriting)

    fn = Text1.Text
    sn = Text2.Text
    em = Text3.Text

    str.WriteLine (fn & " " & sn & " " & em)

End Sub
  
```

4. Add the following code in **Terminate** event of form.

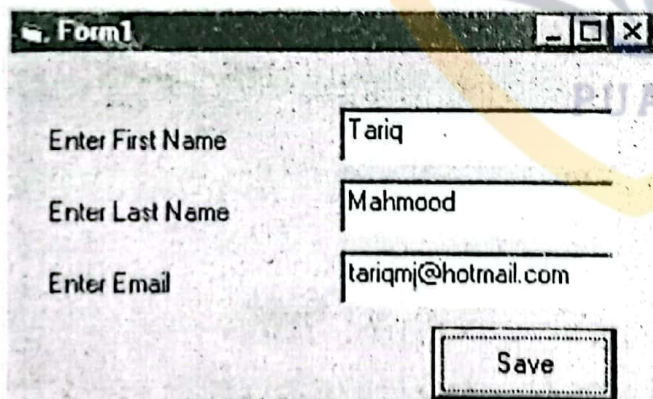


```

Project1 - Form1 (Code)
(General) (Declarations)

Private Sub Form_Terminate()
    str.Close
End Sub
  
```

5. Run the project.
 6. Enter any values in the textboxes and click **Save** button. The entered information will be saved in "C:\Record.txt" file.



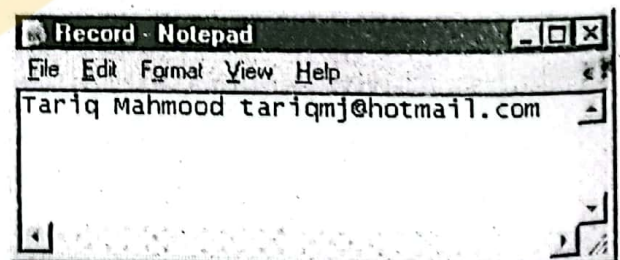
Form1

Enter First Name: Tariq

Enter Last Name: Mahmood

Enter Email: tariqmj@hotmail.com

Save



Record - Notepad

File Edit Format View Help

Tariq Mahmood tariqmj@hotmail.com

How It Works?

The code in Load event of the form declares three variables of type FileSystemObject, File and TextStream. When the button is clicked:

- `FSO.CreateObject("C:\Record.txt")` statement creates a file Record.txt on C drive
- `Set f = FSO.GetFile("C:\Record.txt")` statement links the File object f with Record.txt
- `Set str = f.OpenAsTextStream(ForWriting)` statement links the TextStream object str with File object f and opens the file for writing.

Open Mode	Description
ForReading	Opens the file only for reading.
ForWriting	Opens the file for writing. If file contains some data, it will be overwritten.
ForAppending	Opens the file to write at the end of current contents.

- `str.WriteLine(fn & " " & sn & " " & em)` statement stores the values of the parameters in the text file `Record.txt`.

13.8 Reading Sequential Files

You can also read sequential files to retrieve the information stored in files. The following example demonstrates how you can read files on the disk.

Program 13.3

Write a program that reads data from a sequential file and displays it on the form.

1. Start a new Standard EXE project.
2. Add Microsoft Scripting Runtime Library from Project > References.
3. Design the form as follows:



4. Add the following code in General declaration section.

```
Project1 - Form1 (Code)
  (General) | (Declarations)
  Dim FSO As New FileSystemObject
  Dim str As TextStream
```

5. Add the following code in Click event of the button.

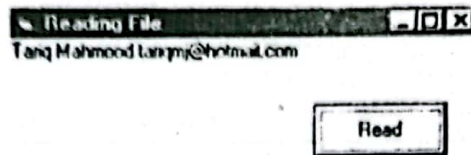
```
Project1 - Form1 (Code)
  Command2 | Click
  Private Sub Command2_Click()
    Dim s As String

    Set f = FSO.GetFile("C:\Record.txt")
    Set str = f.OpenAsTextStream(ForReading)
    s = str.ReadAll
    Print s
  End Sub
```

6. Add the following code in Terminate event of the form.

```
Project1 - Form1 (Code)
  Form | Terminate
  Private Sub Form_Terminate()
    str.Close
  End Sub
```

7. Run the project and click Read button. The contents of the fill will be displayed on the form as follows:



13.8.1 Drawback of Sequential Files

The main disadvantage of sequential files is that you cannot update records easily. The records in these files are not stored with equal memory space. So it is very difficult to find where one record ends and where the next record starts. The current record may be overwritten when updating. Sometime, you need to rewrite the whole file to update one or a few records in the file.

13.9 Random Access Files

Random Access files are implemented by using fixed-length records. Data may be inserted into the file without destroying any other data in the file. Data may also be modified or deleted without rewriting the whole file again.

13.9.1 Reading and Writing Random Access Files

Random access files may be opened as "read only", "write only" or "read and write". If no mode is specified, the default is read and write.

Open filename For Random Access Read Write As #1 Len = recordLength

The **As** clause specifies a file number that will be used as a handle to reference the file. **Get** function is used to read a record from a text file into a user defined record. **Put** function is used to write a user defined record to a file.

The **Get** function takes the file handle, the record number, and the user defined record as parameters. The following example gets record 8 from file handle 1 and places it in a user-defined record.

Get #1, 8, userRecord

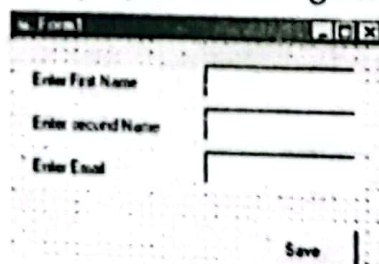
The **Put** function takes the file handle, the record number, and the user defined type as parameters. The following example writes the data stored in the user-defined type to record 4 in file handle 1.

Put #1, 4, userRecord

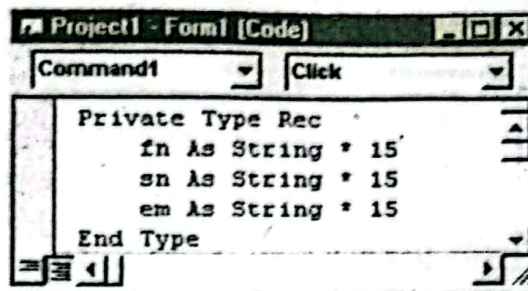
Program 13.4

Write a program that inputs first name, second name and email in text boxes and stores this data in a file using random access technique.

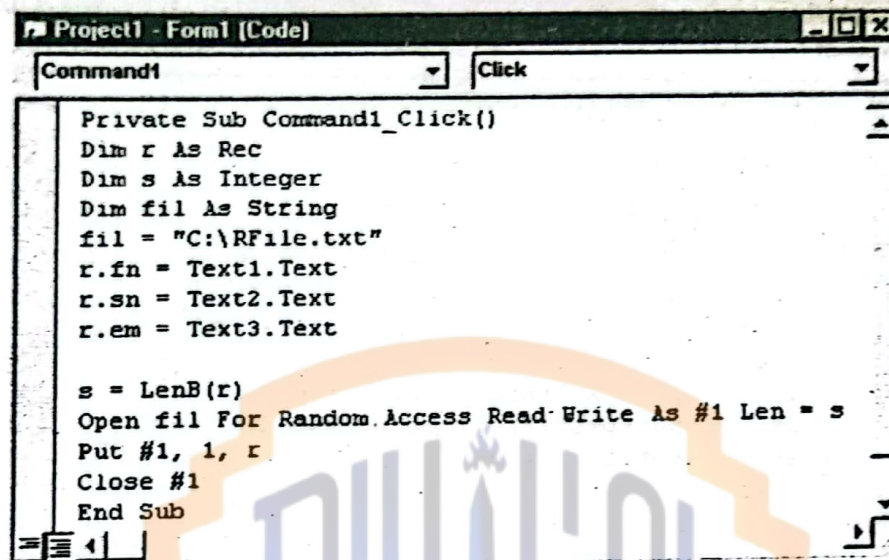
1. Start a new Standard EXE project and design the form as follows:



2. Add the following code in General Declaration section.



3. Add the following code in Click event of the button.



4. Run the project.
5. Enter values in textboxes and press Save button. The record will be saved.

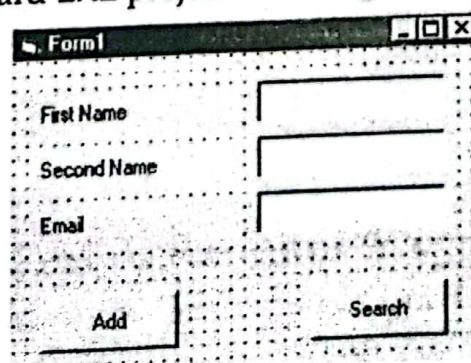
13.9.2 Searching a Record in Random Access File

The following example uses two buttons i.e. Add and Search. When the user click Add button, it get five records from the user and stores them in a file using Random Access File technique. Search button gets the first name and searches the file for that name. If a record with the same First Name is found, it is displayed on the text boxes.

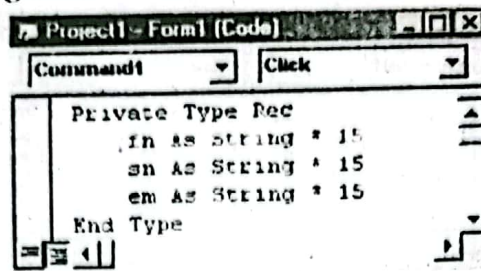
Program 13.5

Write a program that inputs first name, second name and email in text boxes and stores this data in a file using random access technique. The program also searches the data from the file on the basis of first name entered by the user.

1. Start a new Standard EXE project and design the form as follows:



2. Add the following code in General Declaration section.

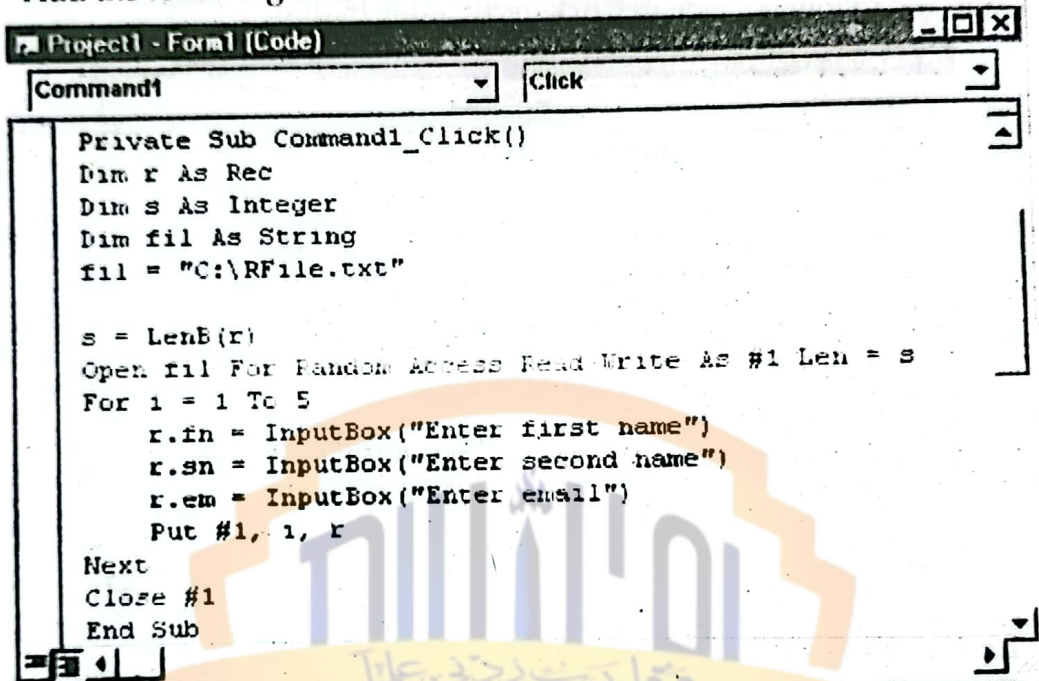


```

Project1 - Form1 (Code)
Command1 Click
Private Type Rec
    fn As String * 15
    sn As String * 15
    em As String * 15
End Type

```

3. Add the following code in Click event of Add button.



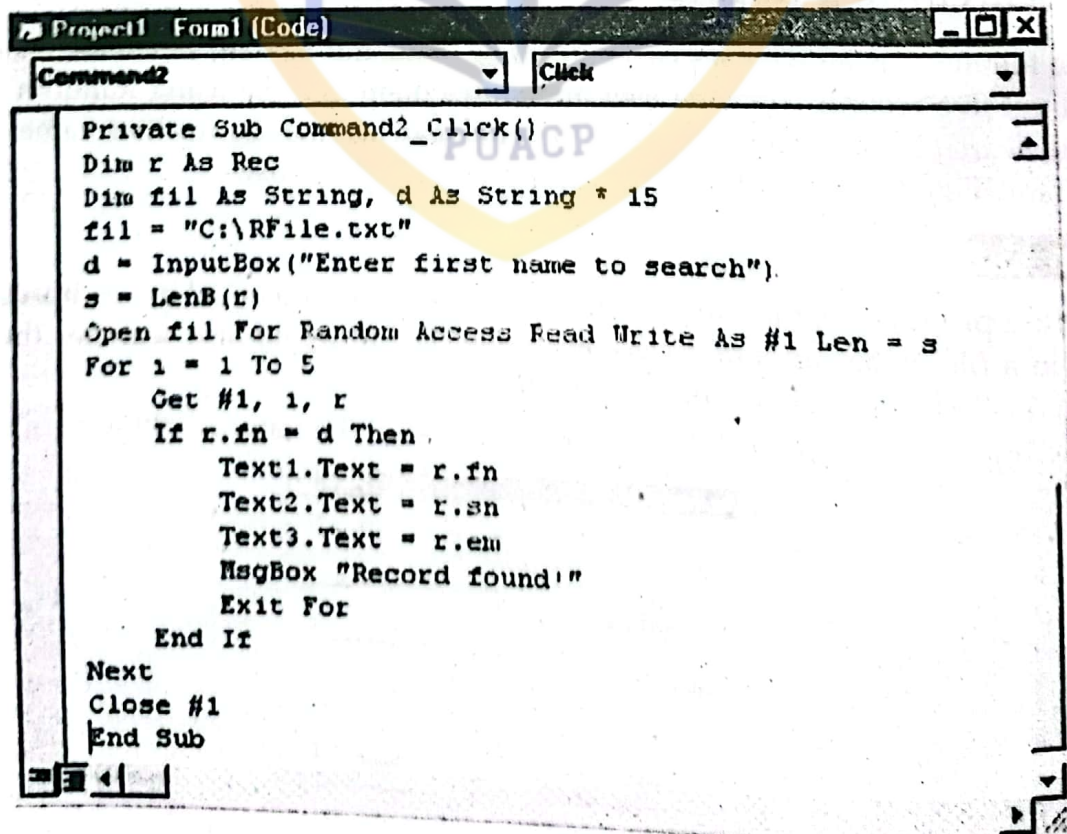
```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1_Click()
    Dim r As Rec
    Dim s As Integer
    Dim fil As String
    fil = "C:\RFile.txt"

    s = LenB(r)
    Open fil For Random Access Read Write As #1 Len = s
    For i = 1 To 5
        r.fn = InputBox("Enter first name")
        r.sn = InputBox("Enter second name")
        r.em = InputBox("Enter email")
        Put #1, i, r
    Next
    Close #1
End Sub

```

4. Add the following code in the Click event of Search button.

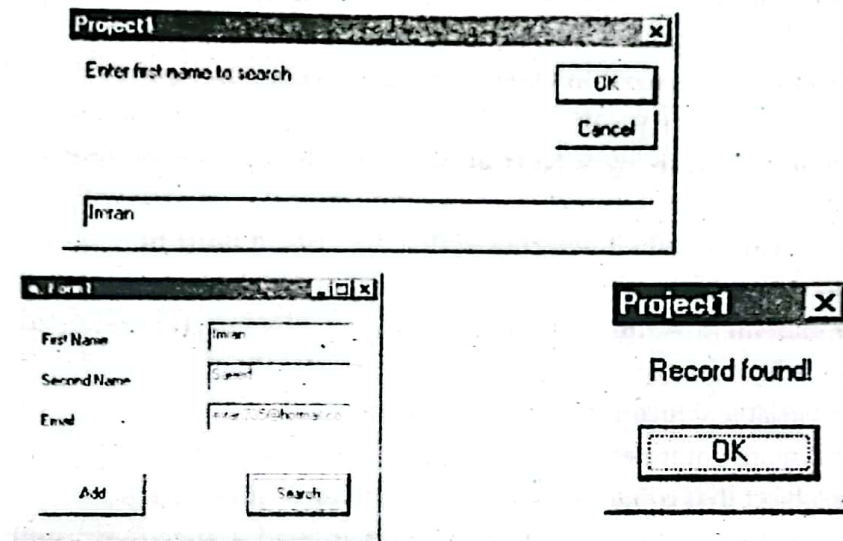


```

Project1 - Form1 (Code)
Command2 Click
Private Sub Command2_Click()
    Dim r As Rec
    Dim fil As String, d As String * 15
    fil = "C:\RFile.txt"
    d = InputBox("Enter first name to search")
    s = LenB(r)
    Open fil For Random Access Read Write As #1 Len = s
    For i = 1 To 5
        Get #1, i, r
        If r.fn = d Then
            Text1.Text = r.fn
            Text2.Text = r.sn
            Text3.Text = r.em
            MsgBox "Record found!"
            Exit For
        End If
    Next
    Close #1
End Sub

```

5. Run the project.
6. Click Add button and give five records to store in the file.
7. Click Search button and enter any First Name to search. If the name is found in the file, the record will be displayed in the textboxes.



Short Questions

Q.1. What is the difference between project file and data file?

A project file is a small text file that contains the names of other files in project. It also contains some information about VB environment. It is saved with .vbp extension. A data file contains actual data such as names and addresses, inventory amounts, account balances etc.

Q.2. List and explain the file modes for data files.

Output File Mode: Data is output from the project and written on the disk. New data is written at the beginning of the file, overwriting any existing data.

Input: Data is input in project from disk. This mode reads data stored on the disk.

Append: Data is output from project and written on disk. Data is added to the end of file.

Random: Data can be input or output and records may be accessed in any order.

Q.3. What is the significance of a file number?

When a file is opened, it must be assigned a file number. Multiple files can be opened at the same time. It is important that each file must be assigned a unique file number. Files are accessed and manipulated using file number rather than file name.

Q.4. Explain the differences between Output and Append modes.

Output and Append modes are used to output data from the project. With Output mode, new data is written at the beginning of file. It overwrites any existing data. Append mode adds new data to the end of file.

Q.5. What are the differences between a random file and a Sequential file?

The primary difference between sequential files and random files is that you may read and write data in any order in a random file. With sequential files, it is always necessary to start at the beginning of the file and proceed in order through the file. Random files are accessed speedily. Each record in a random file is of same size. Sequential files have variable length fields and records.

Multiple Choice

1. Which of the following is NOT an object in file system?
a. Folder b. Drive c. Textstream d. Image
2. Which keyword is used in Open statement to open a sequential file for reading?
a. Input b. Reads c. Write d. Variables
3. Which of the following is NOT an access mode that may be specified in OPEN statement?
a. Read b. Input c. Output d. Append
4. When you are finished working with a data file, it must be:
a. Closed b. Appended c. Output d. Both a and b
5. In the statement Write #1, txtName.Text, the txtName.Text refers to:
a. The record number.
b. The variable which will store the data in txtName
c. The data file number
d. The object that contains the data that will be written to disk.
6. Which of the following method is used to read a specified number of characters starting from the current position in the file:
a. Read b. ReadLine c. Open d. Close
7. Which of the following is used to add data to a file and add an end of line marker:
a. Write b. WriteLine c. Close d. WriteToEnd
8. Which of the following are valid file modes for a sequential file?
a. Append b. Create c. Both a and b d. None
9. Which of the method will close a file?
a. Close b. CloseAll c. Both a and b d. None
10. To create a new record, the file must be opened in:
a. Output mode b. Append mode c. Create mode d. None
11. Which of the following is a correct input statement?
a. Input# 1, strName, curSalary b. Input strName, curSalary As #1
c. Input into #1, strName, curSalary d. Input strName, curSalary into #1
12. Following is not a mode of data file:
a. Open b. Append c. Output d. None

Answers

1. d	2. a	3. a	4. a
5. d	6. a	7. b	8. c
9. a	10. b	11. a	12. a

Fill in the Blanks

1. Visual Basic provides _____ that is used to manipulate files, folders and drives of computer system.
2. To append data to an existing file, open the file in _____ mode.
3. Once access to a file is no longer needed, use _____ statement to break the program's connection to the file.
4. _____ object is used to read and write sequential files.

Answers

1. File System object	2. Append
3. Close	4. TextStream

True / False

1. Random access files are implemented using Fixed-length records.
2. FileStream object writes values directly to a file without another object if it is opened with Write mode.
3. A sequential file can read or write data in any order specified by the developer.
4. A file is a way of storing data only when the application is running.
5. A file can be opened for Input and Output simultaneously.
6. The function Len will determine the length of a file

Answers

1. T	2. F	3. F	4. F
5. T	6. F		

Project 8

Students File Handling

Project Specification

The project **Students File Handling** displays a form that uses textboxes to input the following:

- Roll No
- Name
- Marks

It stores the information in a file when the user click **Save** button. It displays a menu to create a new file or open an existing file. It also displays two buttons to navigate the records stored in the file.

Procedure

1. Start a new Standard EXE project.
2. Place the following controls on the form with the given properties:

Control	Control Name	Property	Value
Label	cmdRollNo	Caption	Roll No
		Alignment	1 - Right Justify
Label	cmdName	Caption	Name
		Alignment	1 - Right Justify
Label	cmdMarks	Caption	Marks
		Alignment	1 - Right Justify
Textbox	txtRollNo	Text	Blank
Textbox	txtName	Text	Blank
Textbox	txtMarks	Text	Blank
CommandButton	cmdPrevious	Caption	<
CommandButton	cmdNext	Caption	>
CommandButton	cmdNew	Caption	New
CommandButton	cmdSave	Caption	Save

3. Place a common dialog box on the form.
4. Design the form as follows:

5. Design the following menu for the form:



6. Add the following code in General Declaration section:

```

Project1 - Form1 (Code)
(General) (Declarations)
Option Explicit
Dim FSO As New FileSystemObject
Dim f As String, rn As Integer
Private Type Rec
    r As Integer
    n As String
    m As Integer
End Type
Dim myRec As Rec
  
```

7. Add the following code in Click event of New button:

```

Project1 - Form1 (Code)
cmdNew Click
Private Sub cmdNew_Click()
    txtRollNo.Text = ""
    txtName.Text = ""
    txtMarks.Text = ""
End Sub
  
```

8. Add the following code in Click event of Save button:

```

Project1 - Form1 (Code)
cmdSave Click
Private Sub cmdSave_Click()
    Open f For Random Access Read Write As #1 Len = LenB(myRec)
    Get #1, Val(txtRollNo.Text), myRec
    If myRec.r = 0 Then
        myRec.r = txtRollNo
        myRec.n = txtName
        myRec.m = txtMarks
        Put #1, myRec.r, myRec
        MsgBox ("Record saved.")
    Else
        MsgBox ("Roll No already exists.")
    End If
    Close #1
End Sub
  
```

9. Add the following code in Click event of < button:

```

Project1 - Form1 (Code)
cmdPrevious Click
Private Sub cmdPrevious_Click()
    Open f For Random Access Read Write As #1 Len = LenB(myRec)
    If rn = 1 Then
        MsgBox ("You are at first record.")
    Else
        rn = rn - 1
        Get #1, rn, myRec
        Call Fill
    End If
    Close #1
End Sub

```

10. Add the following code in Click event of > button:

```

Project1 - Form1 (Code)
cmdNext Click
Private Sub cmdNext_Click()
    Open f For Random Access Read Write As #1 Len = LenB(myRec)
    rn = rn + 1
    Get #1, rn, myRec
    Call Fill
    Close #1
End Sub

```

11. Add the following code in Click event of New menu option:

```

Project1 - Form1 (Code)
new Click
Private Sub new_Click()
    Dim n As String
    n = InputBox("Enter file name:", "File", "New.txt")
    If FSO.FileExists(App.Path & "\" & n) Then
        MsgBox ("File already exists.")
    Else
        FSO.CreateTextFile App.Path & "\" & n
    End If
End Sub

```

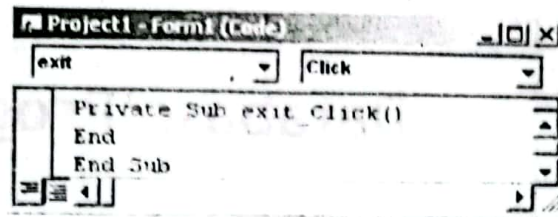
12. Add the following code in Click event of Open menu option:

```

Project1 - Form1 (Code)
open Click
Private Sub open_Click()
    CommonDialog1.ShowOpen
    f = CommonDialog1.FileName
    Open f For Random Access Read Write As #1 Len = LenB(myRec)
    Get #1, 1, myRec
    rn = 1
    Call Fill
    Close #1
End Sub

```

13. Add the following code in Click event of Exit menu option:



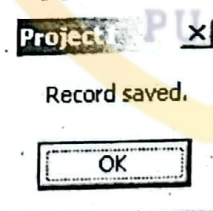
14. Run the project. The form will appear as follows:

15. Click **File > New**. The input box will appear.
 16. Click **OK** button. A text file **New.txt** will be created in the current folder.
 17. Click **File > Open**. Open dialog box will appear.
 18. Select the **New.txt** file.
 19. Enter the following data in textboxes and click **Save** button:

Roll No	1
Name	Usman
Marks	100

Buttons: < New Save >

20. The following message will appear:



21. Click **New** button. All textboxes will become blank.
 22. Add the following data and click **Save** button:

Roll No	2
Name	Adnan
Marks	100

Buttons: < New Save >

23. The record will be saved.
 24. Click **<** button. The first record will be displayed again.